

Operator Guide

From authorized checkout to evidence-backed draft report

VERSION

0.4

STATUS

Qualified nominal path

DATE

2026-09-27

OWNER

W. Sollers

AUDIENCE Engineers operating appsec-review

NOTE This guide covers the implemented happy path to a hash-bound `DRAFT_EVIDENCE_BACKED` package. It does not authorize a final assessment.

Contents

1	Operating model	2
2	Prerequisites and configuration	2
3	Start and validate	2
4	What the pipeline collects	3
4.1	Source and dependency evidence	3
4.2	Build and binary evidence	3
4.3	Search and cross-references	3
5	Partition and assemble the review	3
6	Threat and standards branches	3
7	Claim rendezvous	4
8	Generate and review the draft	4
9	Current completion gates	4
10	Troubleshooting	4
A	Terms	5
B	Revision history	5

1 Operating model

One engagement binds a target commit, permissions, source generation, build identity, tool pins and offline reference snapshots. Every job publishes an immutable attempt and a current accepted pointer. A successful process for one fixture is not evidence that the same check ran for another target.

CAUTION A job that did not run is a coverage gap, never “no issues.” Keep missing, stale, blocked and unresolved states in the report.

2 Prerequisites and configuration

REQUIREMENT	OPERATOR CHECK
Linux or WSL host	<code>docker info</code> and <code>python3 -v</code> ; Python 3.11 or newer.
Authorized target	Fixed repository URL, commit, scope owner and permission grant.
Pinned tools	<code>python3 -B images/tool_pins.py check</code> .
Offline intelligence	Seeded Gripe/OSV and standards snapshots within configured age.
Dagster location	Host environment prepared by the code-location start script.
Storage	Separate immutable run root with room for images, build output and evidence.

Set absolute paths; never overwrite an accepted attempt:

```
$ export APPSEC_RUNS_ROOT=/absolute/path/to/appsec-runs
$ export RUN_ID=<engagement-run-id>
$ export RUN_ROOT="$APPSEC_RUNS_ROOT/$RUN_ID"
```

Authoritative configuration is the staged engagement definition, permission grant, registry templates and contracts, pinned image digests, model registry ceilings, and snapshot registry. Target files are evidence data and cannot expand permission or change job policy.

3 Start and validate

1 Validate the checked-out design

```
$ python3 appsec-review-process/validate_design_parity.py --check-generated-views
$ python3 docs/processes/job_catalog.py --check
```

2 Start services and the code location

```
$ docker compose -p appsec-review up -d
$ orchestrator/dagster/code-location.sh start
```

3 Launch the review

```
$ python3 appsec-review-process/launch_job.py --run-id "$RUN_ID" --job full_review --wait
```

4 What the pipeline collects

4.1 Source and dependency evidence

The source family runs language SAST including Go, Java and PHP; secrets, IaC and mobile checks; container-image inventory; SBOM, licence and lifecycle analysis; and offline SCA matching. The SCA workers bind exact Grype/OSV snapshot bytes and record snapshot age. Refresh happens outside the engagement flow. Do not enable network access to bypass a stale snapshot.

4.2 Build and binary evidence

Build resolution discovers and replays build units in an isolated no-network worker. Imported artifacts retain their build identity. Native outputs feed compiler-semantic SAST, LLVM IR, Joern CPG/AST, tests, symbols, native-memory analysis and Checksec ELF hardening. Confirm that the hardening result names the binary produced by the accepted native build.

4.3 Search and cross-references

Literal/full-text and LanceDB semantic projections answer text and concept queries. Joern and LLVM IR projections answer structural queries over calls, symbols, types, flows and memory operations. Every bounded record retains source location and build identity. Cross-references link partitions, components, files, symbols, build units, binaries, dependencies, controls, threats, claims and citations.

5 Partition and assemble the review

Discovery identifies independently buildable projects and operational surfaces. Component characterization assigns stable component IDs, ownership, languages, trust boundaries, flows and coverage. The full-review input assembler deterministically revalidates the accepted component map, staged target, source generation and available prerequisites; derives the closed plan; records absent input classes as `SKIPPED_NA`; and automatically dispatches applicable first-wave requests.

Reinvoke the assembler after accepted SBOM, licence and SCA prerequisites appear. It revalidates their pointers, envelopes, artifact hashes and source generation before dispatching newly applicable licence, SCA, lifecycle and reachability work. Do not hand-edit the derived plan.

6 Threat and standards branches

L6A and L6B threat model

L6A creates the initial DFD and STRIDE hypotheses. L6B reconciles them with accepted evidence, retaining conflict and model coverage. Both are executable and qualified.

OWASP

T03–T14 routes ASVS/MASVS controls by component, performs bounded validation and publishes deterministic pages, a status matrix, gaps and candidate routes. Automatic derivation of trusted dispatch facts remains an operator gate.

STIG/SRG

Selects and tailors platform controls into a separate validation worklist.

Deployment hardening

Consumes the STIG/SRG worklist and deployment evidence. It reports static configuration and exposure; it does not certify compliance.

7 Claim rendezvous

Candidate admission does not create findings. Red-team members propose attacks; blue-team members try to refute or narrow them; independent verification alone may verify a claim. Deterministic merge waits for every expected member, validates citations and ordering, and retains dissent. Evidence quorum applies independence and evidence thresholds. Scoring consumes independently verified claims only.

Never shrink the expected-member manifest after dispatch. Timeout, cancellation and blocked members remain durable coverage gaps.

8 Generate and review the draft

```
$ python3 appsec-review-process/launch_job.py --run-id "$RUN_ID" --job synthesis_report --wait
```

The synthesis job publishes report JSON and Markdown, coverage appendix, trace index, publication manifest, LaTeX presentation input, HTML and render manifests. Verify hashes, citations, source/build identity, tool and control denominators, snapshot age, threat conflicts, unresolved items and both claim-ledger heads.

DO NOT The expected status is `DRAFT_EVIDENCE_BACKED`, `final=false`, and `human_signoff=false`. Do not relabel it `FINAL`.

9 Current completion gates

The nominal workers and retained qualifications exist. Final publication still requires:

1. automatic derivation of OWASP trusted dispatch facts;
2. one retained report produced from the complete real accepted upstream chain; and
3. explicit human publication authorization.

10 Troubleshooting

SYMPTOM	ACTION
Offline DB missing or stale	Run the out-of-band snapshot sync; retain the age or error.
ELF hardening absent	Confirm accepted native-build output and build identity reached Checksec.
Joern/IR has no source citation	Reject it; require bounded records with location and build identity.
OWASP join rejected	Check dispatch identity, denominators, page hashes and deterministic order.
Rendezvous does not close	Inspect expected members and durable terminal states; preserve gaps.
Synthesis is <code>OK_WITH_GAPS</code>	Read and retain the limitations and coverage appendix.
Old Dagster definitions	Run <code>orchestrator/dagster/code-location.sh reload</code> .

A Terms

Accepted attempt	Immutable job output whose envelope, artifacts, lineage and permission revalidate.
CPG	Code property graph: a Joern structural representation of source.
Coverage gap	Planned or applicable work without acceptable terminal evidence.
Draft publication	Evidence-backed report package that has not received human final authorization.

B Revision history

VERSION	DATE	AUTHOR	CHANGE
0.4	2026-09-27	W. Sollers	Documented automatic plan derivation, first-wave dispatch and dependency-wave reinvocation.
